

Boucles PHP

Dans ce chapitre, vous apprendrez à répéter une série d'actions à l'aide de boucles en PHP.

Différents types de boucles en PHP

Les boucles sont utilisées pour exécuter le même bloc de code encore et encore, tant qu'une certaine condition est remplie. L'idée de base derrière une boucle est d'automatiser les tâches répétitives dans un programme pour économiser du temps et des efforts. PHP prend en charge quatre types de boucles différents.

- **while** - boucle dans un bloc de code tant que la condition spécifiée est vraie.
- **do...while** — le bloc de code exécuté une fois, puis la condition est évaluée. Si la condition est vraie, l'instruction est répétée tant que la condition spécifiée est vraie.
- **for** — parcourt un bloc de code jusqu'à ce que le compteur atteigne un nombre spécifié.
- **foreach** — parcourt un bloc de code pour chaque élément d'un tableau.

Vous apprendrez également à parcourir les valeurs de **array** en utilisant **foreach()** à la fin de ce chapitre. La boucle `foreach()` fonctionne spécifiquement avec des tableaux.

PHP Boucle while

L'instruction **while** parcourt un bloc de code tant que la condition spécifiée dans l'instruction **while** est vraie.

```
while (condition){  
    // Code à exécuter  
}
```

L'exemple ci-dessous définit une boucle qui commence par $\$i=1$. La boucle continuera à s'exécuter tant que $\$i$ est inférieur ou égal à 3. Le $\$i$ augmentera de 1 à chaque exécution de la boucle :

```
<?php  
$i = 1;  
while($i <= 3){  
    $i++;  
    echo "Le nombre est " . $i . "<br>";  
}  
?>
```

PHP Boucle do...while

La boucle **do-while** est une variante de la boucle **while**, qui évalue la condition à la fin de chaque itération de la boucle. Avec une boucle do-while, le bloc de code est exécuté une fois, puis la condition est évaluée. Si la condition est vraie, l'instruction est répétée tant que la condition spécifiée évaluée est vraie

```
do {  
    // Code à exécuter  
}  
while (condition);
```

L'exemple suivant définit une boucle qui commence par \$i=1. Il augmentera ensuite \$i de 1 et imprimera la sortie. Ensuite, la condition est évaluée et la boucle continuera à s'exécuter tant que \$i est inférieur ou égal à 3.

```
<?php  
$i = 1;  
do{  
    $i++;  
    echo "Le nombre est " . $i . "<br>";  
}  
while($i <= 3);  
?>
```

Différence entre while et do...while

La boucle **while** diffère de la boucle **do-while** d'une manière importante - avec une boucle **while**, la condition à évaluer est testée au début de chaque itération de boucle, donc si l'expression conditionnelle est évaluée à faux, la boucle ne sera jamais exécutée.

Avec une boucle **do-while**, en revanche, la boucle sera toujours exécutée une fois, même si l'expression conditionnelle est fausse, car la condition est évaluée à la fin de l'itération de la boucle plutôt qu'au début.

PHP boucle For

La boucle **for** répète un bloc de code tant qu'une certaine condition est remplie. Il est généralement utilisé pour exécuter un bloc de code un certain nombre de fois.

```
for (initialisation; condition; incrémentation){  
    // Code à exécuter  
}
```

Les paramètres de la boucle for ont les significations suivantes :

- **initialisation**— il sert à initialiser les variables du compteur, et est évalué une fois sans condition avant la première exécution du corps de la boucle.
- **condition**— au début de chaque itération, la condition est évaluée. S'il est évalué à true, la boucle continue et les instructions imbriquées sont exécutées. S'il est évalué à false, l'exécution de la boucle se termine.
- **incrémentation**— il met à jour le compteur de boucle avec une nouvelle valeur. Il est évalué à la fin de chaque itération.

L'exemple ci-dessous définit une boucle qui commence par \$i=1. La boucle se poursuivra jusqu'à ce que \$i soit inférieur ou égal à 3. La variable \$i augmentera de 1 à chaque exécution de la boucle :

```
<?php
for($i=1; $i<=3; $i++){
    echo "Le nombre est " . $i . "<br>";
}
?>
```

PHP Boucle foreach

La boucle **foreach** est utilisée pour parcourir des tableaux.

```
foreach ($array as $value){  
    // Code à exécuter  
}
```

L'exemple suivant illustre une boucle qui imprimera les valeurs du tableau donné :

```
<?php  
$colors = array("Red", "Green", "Blue");  
foreach($colors as $value){  
    echo $value . "<br>";  
}  
?>
```


Il existe une autre syntaxe de la boucle foreach, qui est l'extension de la première.

```
foreach ($array as $key => $value){  
    // Code à exécuter  
}
```

```
<?php  
$superhero = array(  
    "name" => "Peter Parker",  
    "email" => "peterparker@mail.com",  
    "age" => 18  
);  
  
foreach($superhero as $key => $value){  
    echo $key . " : " . $value . "<br>";  
}  
?>
```